

Voku CMS Nasıl çalışır?

.NET 10 · ASP.NET Core MVC
Entity Framework Core · SQLite · Razor

Kuruluştan veri akışına, controller'dan forma ve veritabanından
ekrana adım adım.

Türkçe eğitim belgesi · 24 Eylül 2026
Kod referansı: 2688cd4
Mevcut durum: admin giriş kontrolü kapalı

İçindekiler

- 01 Proje ne yapıyor?
- 02 Diller ve teknolojiler
- 03 Klasörleri okuyarak projeyi tanımak
- 04 Proje nasıl kuruldu?
- 05 Mevcut projeyi çalıştırmak
- 06 Program.cs: uygulamanın başlangıç noktası
- 07 Bir HTTP isteğinin geçtiği yol
- 08 Models: veriyi temsil eden sınıflar
- 09 Doğrulama: formdan gelen veriye güvenmemek
- 10 DbContext ve SQLite
- 11 Migration dosyaları ne işe yarar?
- 12 Yeni bir alan ekleme alıştırması
- 13 SeedData: HTML'den ilk içeriklere
- 14 SiteController: ziyaretçiye veri sunmak
- 15 Page.cshtml: entity'nin ekrana dönüşmesi
- 16 AdminController ve tüm route'lar

Bölüm isimleri tıklanabilir. Kod kaynağı bağlantıları anlatılan sürümün ilgili dosyasını açar. Kod blokları aksi belirtilmedikçe mevcut uygulamayı anlatır.

İçindekiler · devam

17 Admin view'ları ve layout ilişkisi

18 Bir admin formunun anatomisi

19 Örnek 1: Hakkımızda sayfasını düzenlemek

20 Örnek 2: yeni yazı ve slug değişikliği

21 editor.js: görsel alanlar nasıl oluşuyor?

22 Ziyaretçi formları ve statik varlıklar

23 Giriş kontrolü ve mevcut güvenlik davranışı

24 Testler: davranışı nasıl doğruluyoruz?

25 GitHub, CI ve yayınlama

26 Hata ayıklama ve sık karşılaşılan durumlar

27 Uygulamalı çalışma planı ve cevaplar

28 Terimler ve hızlı dosya rehberi

Bölüm isimleri tıklanabilir. Kod kaynağı bağlantıları anlatılan sürümün ilgili dosyasını açar. Kod blokları aksi belirtilmedikçe mevcut uygulamayı anlatır.

Proje ne yapıyor?

Öğrenme hedefi: Siteyi yalnızca görünen ekranlardan değil, verinin kaynağından başlayarak tanımak.

Voku CMS, Voku HTML tasarımını kullanan bir blog ve sayfa yönetim uygulamasıdır. Ziyaretçi yayınlanmış sayfaları okur. Yönetim ekranında aynı içerikler düzenlenir, yeni blog detayları oluşturulur ve taslak/yayın durumu değiştirilir. Sayfa içerikleri SQLite veritabanındaki Pages tablosunda saklanır.

Bu belge, **2688cd4** commitindeki uygulamayı anlatır. Kod örneklerinin bir bölümü okunabilirlik için satırlara ayrılmıştır; sadeleştirilmiş veya öneri niteliğindeki örnekler ayrıca belirtilmiştir. Belge üretimi uygulamanın veritabanını okumaz ve kişisel şifreleri içermez.

Mevcut davranış	Uygulamadaki karşılığı
Ziyaretçi arayüzü	Voku CSS, görseller, JavaScript eklentileri ve Views/Site/Page.cshtml
Yönetim arayüzü	/admin altında içerik listesi ve düzenleme formları
Veri saklama	EF Core aracılığıyla SQLite; varsayılan dosya App_Data/voku.db
İlk içerikler	9 sabit sayfa + 4 örnek blog detayı = 13 kayıt
Detay adresi	/blog/{slug}; örnek: /blog/heyecanli-bir-yolculuk
Giriş kontrolü	Şimdilik kapalı: Admin:RequireAuthentication=false
Ziyaretçi formları	Gönderim kapalı; e-posta göndermez, veri kaydetmez

En önemli ayrım: Ekranın tasarımı HTML/CSS ile, o ekrana hangi verinin gideceği C# controller ile, verinin kalıcılığı EF Core + SQLite ile sağlanır.

Admin formları ile ziyaretçi formları aynı amaçta değildir. Admin formunun POST isteği içerik kaydeder. Ziyaretçinin iletişim/bülten formunun gönderimi ise tarayıcıda durdurulur. Bu iki davranış ilerleyen bölümlerde ayrı izlenecektir.

Kod kaynağı: [README.md:1](#)

Diller ve teknolojiler

Öğrenme hedefi: Hangi teknoloji nerede çalışıyor ve neden var?

Teknoloji	Çalıştığı yer / görevi	Bu projeden örnek
C#	Sunucuda iş akışı, sorgu ve doğrulama	AdminController.Save, ContentPage
ASP.NET Core MVC	HTTP isteklerini controller ve view ile işler	.NET 10 üzerinde çalışan web çatısı
Razor / .cshtml	Sunucuda C# değerlerini HTML içine yerleştirir	@model ContentPage, @Model.Title
HTML	Tarayıcının sayfa yapısı	form, input, article, h1
CSS	Görünüm ve ekran boyutuna uyum	style.css, responsive.css, admin.css
JavaScript	Tarayıcı etkileşimi	editor.js HTML düzenler; cms.js formu durdurur
SQL / SQLite	Kalıcı veri ve tablolar	Pages tablosu, UNIQUE indeks
EF Core	C# nesneleri ile veritabanı arasında eşleme	db.Pages ve SaveChangesAsync
LINQ	C# içinde sorgu ifadesi	Where, OrderBy, SingleOrDefaultAsync
JSON	Uygulama ve araç ayarları	appsettings.json, launchSettings.json
XML	Proje derleme tanımı	Voku.Web.csproj
Python	HTTP entegrasyon testi	tests/smoke.py
YAML	GitHub Actions iş akışı	.github/workflows/ci.yml
Git / GitHub	Sürüm takibi ve uzak depo	feat, fix, test, docs commitleri

C#, SQL ve JavaScript farklı ortamlarda çalışır. JavaScript doğrudan SQLite dosyasına bağlanmaz. Formu sunucuya gönderir; C# kodu doğrular ve EF Core üzerinden kaydeder. Razor kodu da tarayıcıya C# olarak gönderilmez: sunucuda çalışıp HTML üretir.

SQLite, SQL dilini kullanan dosya tabanlı bir veritabanı motorudur. Bu projede Microsoft SQL Server servisi yoktur. EF Core ise veritabanının kendisi değildir; kullanılan sağlayıcı `Microsoft.EntityFrameworkCore.Sqlite` olduğu için sorgular SQLite için yürütülür.

jQuery ve Bootstrap Voku paketindeki ön yüz araçlarıdır. React, Angular veya ayrı bir SPA uygulaması bulunmaz. Python da web sunucusunun bir parçası değildir; testi çalıştırdığınızda kullanılır.

Kod kaynağı: [Voku.Web/Voku.Web.csproj:1](#)

Klasörleri okuyarak projeyi tanımak

Öğrenme hedefi: Bir değişiklik yapacağınızda doğru dosyayı bulmak.

```
dotnet-blogpage/
├─ Voku.slnx                # Çözüm: projeleri bir arada tutar
├─ Voku.Web/
│   ├─ Program.cs           # Başlatma ve HTTP işleme hattı
│   ├─ Controllers/         # HTTP endpointleri
│   ├─ Models/              # Entity ve form modelleri
│   ├─ Data/                # DbContext, seed, tasarım zamanı factory
│   ├─ Migrations/          # Veritabanı şema geçmişi
│   ├─ Views/               # Razor ekranları ve layoutlar
│   ├─ Content/             # İlk içerik için HTML kaynakları
│   ├─ wwwroot/             # Tarayıcıya sunulan statik dosyalar
│   ├─ App_Data/            # Çalışma sırasında oluşan SQLite dosyası
│   └─ Properties/launchSettings.json
├─ Voku-HTML-Package/       # Orijinal tasarım referansı
├─ tests/smoke.py           # Uçtan uca HTTP akış testleri
├─ .github/workflows/ci.yml # Otomatik build ve test
├─ dotnet-tools.json        # dotnet-ef araç sürümü
└─ README.md               # Kısa çalıştırma rehberi
```

Dosya/dizin	Ne zaman kullanılır?
Content/*.html	Yalnızca Pages tablosu boşken başlangıç içeriği olarak okunur.
wwwroot/images	Tarayıcı /images/... isteği yaptığı anda dosya olarak sunulur.
Views/*.cshtml	Controller bir ViewResult döndürdüğünde HTML oluşturur.
bin ve obj	Derleme çıktıları ve ara dosyalar; kaynak dosyalar değildir.
App_Data/voku.db	Gerçek içerik verisi; .gitignore nedeniyle commitlenmez.

Content/about.html dosyasını değiştirmek, mevcut veritabanındaki Hakkımızda içeriğini otomatik güncellemez. İlk kurulum sonrası içerik için /admin/pages/about/edit kullanılmalıdır.

Varsayılan MVC şablonundan kalan Views/Home/Index.cshtml, Privacy.cshtml, Shared/_Layout.cshtml, Shared/Error.cshtml, ErrorViewModel ve _ValidationScriptsPartial mevcut aktif akışın ekranları değildir. HomeController kaldırılmıştır. Bir dosyanın projede bulunması, isteklerin onu kullandığı anlamına gelmez. Aktif admin layoutu _AdminLayout; aktif ziyaretçi view'ı Site/Page'dir.

Kod kaynağı: [Voku.Web/Views/_ViewStart.cshtml:1](#)

Proje nasıl kuruldu?

Öğrenme hedefi: Hazır şablondan çalışan CMS'ye geçişin adımlarını anlamak.

Aşağıdaki komutlar kuruluş mantığını öğretir. Mevcut klasörde tekrar `dotnet new` çalıştırmanız gerekmez; bunlar projenin baştan nasıl oluşturulabileceğini gösterir.

```
dotnet new mvc -n Voku.Web --no-restore
dotnet new sln -n Voku
dotnet sln Voku.slnx add Voku.Web/Voku.Web.csproj

dotnet add Voku.Web package Microsoft.EntityFrameworkCore.Sqlite --version 10.0.12
dotnet add Voku.Web package Microsoft.EntityFrameworkCore.Design --version 10.0.12

dotnet new tool-manifest
dotnet tool install dotnet-ef --version 10.0.12
```

1. MVC şablonu C# web projesi, Razor yapısı ve başlangıç ayarlarını üretir. .slnx dosyası projeyi bir çözüm altında toplar.
2. SQLite paketi çalışma sırasında veritabanına erişimi sağlar. Design paketi migration gibi geliştirme zamanı işlemleri içindir; PrivateAssets=all bağımlılığın tüketen projelere yayılmasını sınırlar.
3. Voku CSS, font, görsel ve JavaScript dosyaları wwwroot altına taşınır. Orijinal HTML sayfaları Content altında seed kaynağı olur.
4. ContentPage ve AdminUser modellenir, SiteDbContext oluşturulur. İlk migration ile tablolar tarif edilir.
5. SiteController, AdminController ve Razor view'ları yazılır. SeedData HTML gövdelerini veritabanına aktarır.
6. Entegrasyon testleri ve GitHub Actions eklenir. Anlamlı aşamalar ayrı commitlerle kaydedilir.

Voku.Web.csproj içindeki `TargetFramework=net10.0` hedef platformu belirtir. `Nullable=enable` null olabilen referansları ayırt etmeye yardımcı olur. `ImplicitUsings` sık kullanılan namespace'lerin bazılarını otomatik ekler. Bunlar veritabanı ayarları değil, C# derleme ayarlarıdır.

Content dosyaları csproj içindeki `CopyToOutputDirectory` ve `CopyToPublishDirectory` ile çıktıya kopyalanır. Böylece yayınlanmış uygulama da boş veritabanına başlangıç içeriklerini yükleyebilir.

Kod kaynağı: [Voku.Web/Voku.Web.csproj:1](#)

Mevcut projeyi çalıştırmak

Öğrenme hedefi: Terminal konumu, profil ve ayarların etkisini ayırmak.

```
# Her klasörden çalışabilecek mutlak yol:
dotnet run --project /Users/macuser/Desktop/dotnet-blogpage/Voku.Web/Voku.Web.csproj --launch-profile http

# Repo kök klasöründeyse:
dotnet restore Voku.slnx
dotnet build Voku.slnx
dotnet run --project Voku.Web --launch-profile http

# Zaten Voku.Web klasöründeyse:
dotnet run --launch-profile http
```

--project Voku.Web, geçerli terminal konumunun altında Voku.Web arar. Terminal zaten Voku.Web içindeyse yanlışlıkla Voku.Web/Voku.Web yolu aranır. "The provided file path does not exist" hatasının nedeni bu olabilir. Mutlak csproj yolu bu belirsizliği giderir.

Ayar	Etkisi
http profili	http://localhost:5080 adresi; Development ortamı
https profili	https://localhost:7080 ve http://localhost:5080
Admin:RequireAuthentication=false	/admin doğrudan açılır; yeni hesap zorunlu değildir.
ConnectionStrings__Default	Varsayılan SQLite bağlantısı yerine verilen bağlantı kullanılır.
Admin__Username / Admin__Password	Giriş etkin ve hesap yoksa ilk hesap oluşturmak için okunur.

JSON içindeki Admin:RequireAuthentication anahtarının ortam değişkeni karşılığı Admin__RequireAuthentication olur. Çift alt çizgi, iç içe ayar anahtarını temsil eder. Program.cs'de GetValue için verilen true, ayar hiç yoksa kullanılacak değerdir; mevcut appsettings.json ise açıkça false verir.

Şu an beklenen kullanım: uygulamayı başlat → /admin aç → içerik düzenle. Daha önceki "ilk çalıştırma için şifre belirleyin" koşulu, artık yalnızca giriş kontrolü etkin ve AdminUsers boşsa çalışır.

launchSettings.json yerel geliştirme profilleridir. Yayınlanan DLL'yi doğrudan başlatırken bu profilin otomatik uygulanacağını varsaymayın. Yayın ortamının URL, ortam adı ve bağlantı ayarları ayrıca sağlanır.

Kod kaynağı: [Voku.Web/Properties/launchSettings.json:1](#)

Kod kaynağı: [Voku.Web/appsettings.json:1](#)

Program.cs: uygulamanın başlangıç noktası

Öğrenme hedefi: Servis kaydı ile istek işleme hattının farklı görevlerini görmek.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews(o =>
    o.Filters.Add(new AutoValidateAntiforgeryTokenAttribute()));
builder.Services.AddDbContext<SiteDbContext>(o => o.UseSqlite(...));
// ... cookie, yetkilendirme ve hız sınırı servisleri
var app = builder.Build();
// ... middleware kayıtları
app.MapControllers();
await SeedData.InitializeAsync(app.Services, app.Environment, app.Configuration);
app.Run();
```

Bu parça sadeleştirilmiştir. `builder.Services` uygulamanın hangi bileşenleri üretebileceğini bildirir. `AddControllersWithViews` controller ve Razor desteğini ekler. Global antiforgery filtresi POST gibi güvenli olmayan HTTP yöntemlerinde token doğrulaması sağlar. `AddDbContext` veritabanı erişiminin nasıl kurulacağını tanımlar.

Dependency injection (DI), controller'ın ihtiyacı olan `SiteDbContext`'i framework'ün sağlamasıdır. `AdminController(SiteDbContext db, IConfiguration configuration)` yazınca controller'ın içinde bağlantı açan yeni bir context oluşturmayız. İstek kapsamında verilen context kullanılır. Böylece bağlantı yapılandırması tek yerde tutulur.

`Directory.CreateDirectory`, `App_Data` klasörü yoksa oluşturur. Varsayılan veritabanı yolu `ContentRootPath`'e bağlanır; bu sayede çalışma verisi uygulamanın içerik kökünde tutulur. Bağlantı ayarı verilmişse ?? operatörü varsayılan yerine o değeri seçer.

1. `builder.Build()` servis kayıtlarından çalışan uygulama nesnesini oluşturur.
2. `MapControllers()`, controller üzerindeki route niteliklerini endpoint olarak kaydeder.
3. `SeedData.InitializeAsync()` migrationı uygular ve gerekiyorsa ilk verileri ekler.
4. `app.Run()` web sunucusunun istek kabul ettiği çalışmayı başlatır. Seed aşamasında exception olursa `Run`'a ulaşamaz.

`SeedData` içinde `CreateScope` kullanılmasının nedeni, başlangıç kodunda HTTP isteği olmamasıdır. `DbContext` scoped bir servistir; seed için açıkça bir scope açılır ve iş bitince kapatılır. `await`, veritabanı işleminin bitmesini bekler; HTTP sonucu oluşmadan önce gereken verinin hazır olmasını sağlar.

Kod kaynağı: [Voku.Web/Program.cs:1](#)

Bir HTTP isteğinin geçtiği yol

Öğrenme hedefi: Middleware sırasının isteğin sonucunu nasıl belirlediğini öğrenmek.



Program.cs bileşeni	Görevi
UseExceptionHandler / Hsts / HttpsRedirection	Development dışındaki ortamda eklenir; hata ve HTTPS davranışı sağlar.
Özel başlık middleware'i	nosniff ve Content-Security-Policy başlıklarını yanıtı ekler.
UseStaticFiles	wwwroot dosyalarını sunar. /css/style.css için controller gerekmez.
UseRouting	URL ve HTTP yöntemine uyan endpointi belirler.
UseRateLimiter	Örneğin login POST üzerindeki login politikasını uygular.
UseAuthentication	Varsa oturum cookie'sinden kullanıcı kimliğini oluşturur.
UseAuthorization	Endpoint üzerindeki AdminAccess gibi yetki politikasını değerlendirir.

Authentication “kim bu kullanıcı?” sorusunu, authorization “bu işlemi yapabilir mi?” sorusunu cevaplar. Şu anda AdminAccess politikası ayardan herkese izin verir; bu, authentication servisinin projeden kaldırıldığı anlamına gelmez.

GET genellikle ekran okumak, POST form verisi göndermek için kullanılır. Aynı URL iki ayrı action'a gidebilir: GET /admin/posts/new formu açar; POST /admin/posts/new kaydı oluşturur. Yalnızca URL'ye bakmak yeterli değildir; HTTP yöntemi de eşleşmenin parçasıdır.

Kod kaynağı: [Voku.Web/Program.cs:30](#)

Models: veriyi temsil eden sınıflar

Öğrenme hedefi: Entity, form modeli ve hesaplanan alanı birbirinden ayırmak.

ContentPage alanı	Tür / anlam	Nerede kullanılır?
Id	int; kalıcı kayıt kimliği	EF anahtarı; editörde yeni kayıt ayrımı
Title	string; en çok 160 karakter	Admin listesi ve tarayıcı başlığı
Slug	string; en çok 120; küçük harf/rakam/tire	Route sorgusu ve URL üretimi
IsDetail	bool	false: sabit sayfa; true: blog detayı
BodyHtml	zorunlu string	HTML editörü ve ziyaretçi ekranının gövdesi
Description	nullable string; en çok 300	SEO meta açıklaması ve dinamik yazı listesi
Published	bool; başlangıç değeri true	Ziyaretçi sorgusunda yayın filtresi
UpdatedUtc	DateTime	Kaydetmede UTC zaman; yazı listesinde sıralama
Url	hesaplanan string	Linkler; veritabanında sütun değildir

```
public string Url => IsDetail
    ? $"/blog/{Slug}"
    : Slug == "home" ? "/" : $"/{Slug}";
```

Örnek: Slug=about ve IsDetail=false ise Url=/about olur. Slug=ilk-yazim ve IsDetail=true ise Url=/blog/ilk-yazim olur. home sabit sayfası / adresine çevrilir. Url, başka alanlardan hesaplandığı için veritabanına ayrı kaydetmek gereksizdir.

AdminUser, Id, Username ve PasswordHash alanlarıyla AdminUsers tablosuna eşlenen entity'dir.

LoginModel, Username ve Password içeren giriş formu modelidir; DbSet olarak kayıtlı olmadığı için tablosu yoktur. İkisi aynı dosyada tanımlanmış olsa da amaçları farklıdır.

ContentPage hem EF entity'si hem view/form modeli olarak kullanılıyor. Bu başlangıç projesini sade tutar. Daha büyük bir uygulamada EditPageInput ve PageViewModel gibi ayrı modeller, formun değiştirebileceği alanları daha açık sınırlayabilir. Mevcut Save metodu da gelen nesneyi doğrudan tümüyle güncellemek yerine izin verilen alanları tek tek kopyalar.

Kod kaynağı: [Voku.Web/Models/ContentPage.cs:1](#)

Doğrulama: formdan gelen veriye güvenmemek

Öğrenme hedefi: Tarayıcı doğrulaması, ModelState ve veritabanı kuralının rolünü izlemek.

```
[Required, StringLength(120),
RegularExpression(@"^[a-z0-9]+(?:-[a-z0-9]+)*$"),
ErrorMessage = "Slug küçük harf, rakam ve tire içerebilir.")]
public string Slug { get; set; } = "";
```

Bu kural ilk-yazım, dotnet-10 ve 2026 değerlerini kabul eder. Boşluk, büyük harf, Türkçe karakter, başta/sonda tire ve art arda iki tire uygun değildir. Uygulama Türkçe başlığı kendiliğinden slug'a dönüştürmez; kullanıcı geçerli slug girmelidir.

Katman	Örnek	Neden tek başına yetmez?
HTML	required, maxlength, readonly	Tarayıcı dışından istek gönderilerek aşılabilir.
Model doğrulama	Required, StringLength, RegularExpression	Tek kaydı doğrular; diğer kayıtlarla çakışmayı tek başına çözmez.
Controller	Aynı tür/slug başka kayıta var mı?	Eşzamanlı istekler arasında yarış olabilir.
Veritabanı	UNIQUE(IsDetail, Slug)	Son güvence; kullanıcı dostu mesaj için controller kontrolü de gerekir.

MVC, form alanlarını C# nesnesine bağladıktan sonra doğrulama sonuçlarını ModelState içinde tutar. ModelState.IsValid false ise Save, View("Edit", input) döndürür. Böylece kullanıcının girdiği değerler yeniden gösterilir; işlem veritabanına yazılmaz.

Sabit sayfanın slug'ı yalnızca arayüzde readonly değildir. Sunucuda oldSlug != input.Slug kontrolü de yapılır. Bu, HTML özelliğini kaldırıp farklı slug gönderen bir isteğin de reddedilmesini sağlar. Formdaki asp-validation-summary="All" sunucunun eklediği hata mesajlarını gösterir.

Description C# tarafında null olabilir, ancak EF yapılandırması sütunu zorunlu yapar. Save metodundaki input.Description ?? "" boş açıklamayı boş string olarak kaydeder. SQLite için StringLength'i tek başına kesin veritabanı uzunluk engeli saymayın; burada form/model doğrulaması anlamlıdır.

Kod kaynağı: [Voku.Web/Controllers/AdminController.cs:60](#)

DbContext ve SQLite

Öğrenme hedefi: C# sorgusunun nasıl tablo verisine dönüştüğünü öğrenmek.

```
using Microsoft.EntityFrameworkCore;
using Voku.Web.Models;
namespace Voku.Web.Data;
public class SiteDbContext(DbContextOptions<SiteDbContext> options) : DbContext(options)
{
    public DbSet<ContentPage> Pages => Set<ContentPage>();
    public DbSet<AdminUser> AdminUsers => Set<AdminUser>();
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<ContentPage>().HasIndex(p => new { p.IsDetail, p.Slug }).IsUnique();
        modelBuilder.Entity<ContentPage>().Property(p => p.Description).IsRequired();
        modelBuilder.Entity<ContentPage>().Ignore(p => p.Url);
        modelBuilder.Entity<AdminUser>().HasIndex(p => p.Username).IsUnique();
    }
}
```

`DbSet<ContentPage> Pages`, `Pages` tablosuna sorgu yazacağımız giriş noktasıdır. Liste gibi görünse de her kullanım önceden tüm satırların belleğe alınması anlamına gelmez. Where gibi LINQ işlemleri bir sorgu oluşturur; `ToListAsync` veya `SingleOrDefaultAsync` sonuç istendiğinde veritabanı sorgusu yürütülür.

`OnModelCreating`, sınıfların veritabanına nasıl eşleneceğini ayrıntılandırır. Birleşik benzersiz indeks sayesinde sabit `about` ile `blog/about` birlikte bulunabilir; ama iki `blog/about` bulunamaz. `AdminUsers.Username` için de benzersizlik vardır. `Ignore(p => p.Url)` hesaplanan `Url` alanının sütun olmasını engeller.

```
// Kavramsal eşleşme; EF'nin birebir ürettiği SQL değildir.
var page = await db.Pages.SingleOrDefaultAsync(p =>
    p.Slug == "about" && !p.IsDetail && p.Published);

SELECT * FROM Pages
WHERE Slug = 'about' AND IsDetail = 0 AND Published = 1;
```

Okuma ekranlarında `AsNoTracking()` kullanılır; context bu nesneleri değişiklik takibine almaz. Kaydetme akışında ise kayıt tracking ile yüklenir. Özellikler değiştirilip `SaveChangesAsync` çağrıldığında EF uygun UPDATE işlemini üretir. Yeni nesne için `Add` kullanılması ise INSERT gerektiğini bildirir.

Mevcut şemada iki bağımsız uygulama tablosu bulunur; aralarında foreign key veya yazar ilişkisi yoktur. Bir `ContentPage` hangi admin tarafından düzenlendi bilgisini taşımaz. Bu nedenle bu projede olmayan bir `User-Post` ilişkisini varmış gibi düşünmemek gerekir.

Kod kaynağı: [Voku.Web/Data/SiteDbContext.cs:1](#)

Migration dosyaları ne işe yarar?

Öğrenme hedefi: Model kodu ile gerçek veritabanı şemasının aynı şey olmadığını görmek.

```
ContentPage / AdminUser + OnModelCreating
→ dotnet ef migrations add ...
→ Up / Down + Designer + ModelSnapshot
→ MigrateAsync() / database update → SQLite şeması
```

Dosya / tablo	Anlamı
20260923215013_InitialCreate.cs	Up: AdminUsers ve Pages tablolarını, benzersiz indeksleri oluşturur. Down: iki tabloyu kaldırır.
InitialCreate.Designer.cs	Migrationın ilişkilendirildiği context ve hedef model metaverisi.
SiteDbContextModelSnapshot.cs	Bir sonraki migration karşılaştırmasında kullanılan son model görünümü.
__EFMigrationsHistory	Hangi migrationların uygulanmış olduğunu veritabanında izler.
__EFMigrationsLock	SQLite migration uygulamasını eşgüdümlemek için kullanılan EF altyapı tablosu.

Migration, içerik yazısı veya HTML sayfası değildir. Şemadaki değişikliklerin sürümlenmiş tarifiidir. InitialCreate.Up sütunları ve indeksleri oluşturur; Hakkımızda içeriğini eklemek SeedData'nın görevidir. Böylece “tabloyu oluşturmak” ile “tabloya ilk satırları koymak” ayrılır.

Model sınıfına yeni özellik eklemek mevcut SQLite dosyasına otomatik sütun eklemes. Önce yeni migration üretilir, ardından uygulanır. Bu projede uygulama açılırken MigrateAsync çağrıldığı için üretilmiş ve derlenmiş bekleyen migrationlar başlangıçta uygulanır.

Down metodunu çalıştırmak geri alma işlemidir; mevcut InitialCreate.Down tabloları kaldırdığı için içerik kaybına yol açabilir. Eğitim sırasında denemeler ayrı bir veritabanında yapılmalıdır.

Terminalde CREATE TABLE, INSERT INTO __EFMigrationsHistory veya “Acquiring an exclusive lock” görmek tek başına hata değildir. Bunlar normal migration günlükleri olabilir. Hata değerlendirmesinde son exception'a ve işlemin tamamlanıp tamamlanmadığına bakılır.

Kod kaynağı: [Voku.Web/Migrations/20260923215013_InitialCreate.cs:1](https://voku.com/Web/Migrations/20260923215013_InitialCreate.cs:1)

Yeni bir alan ekleme alıştırması

Öğrenme hedefi: Entity → migration → controller → form → ziyaretçi view zincirini tamamlamak.

Bu bölüm önerilen bir alıştırmadır; AuthorName mevcut projede yoktur. Blog yazılarına yazar adı eklemek istediğinizi düşünün. Yalnızca bir input eklemek yeterli değildir; verinin tüm yolunu tanımlamanız gerekir.

1. ContentPage sınıfına [StringLength(100)] public string? AuthorName { get; set; } ekleyin. Başlangıç için nullable seçmek mevcut kayıtları etkilerken basit bir geçiş sağlar.
2. dotnet ef migrations add AddAuthorName --project Voku.Web çalıştırın. Üretilen Up metodunda Pages tablosuna doğru alanın eklendiğini inceleyin.
3. Migrationı development veritabanına uygulayın. Uygulamanın MigrateAsync başlangıcı veya CLI database update kullanılabilir.
4. Views/Admin/Edit.cshtml içinde AuthorName için asp-for input ekleyin.
5. AdminController.Save içinde page.AuthorName = input.AuthorName; atamasını ekleyin. Bu adım yoksa form alanı model binding ile gelse bile kayda aktarılmaz.
6. Views/Site/Page.cshtml içinde uygun konumda @Model.AuthorName gösterin. Gerekirse yalnızca IsDetail sayfalarda gösterin.
7. Testte yeni değeri POST edin, ziyaretçi yanıtında görünmesini ve yeniden başlatınca korunmasını doğrulayın.

```
dotnet tool restore
dotnet ef migrations add AddAuthorName --project Voku.Web
dotnet ef database update --project Voku.Web
dotnet build Voku.slnx
python3 tests/smoke.py
```

SiteDbContextFactory, EF CLI çalışırken web sunucusunu başlatmadan bir context üretir. Bu sayede migration oluşturmak için admin şifresi veya seed işlemi gerekmez. Factory varsayılan olarak görelî App_Data/voku.db bağlantısı kullanır; Program.cs ise ContentRootPath tabanlı varsayılan yol kurar. CLI ve uygulamada aynı veritabanını hedeflediğinizi kontrol etmek gerekir; ConnectionStrings__Default ikisine de açık bağlantı verebilir.

Schema değişikliği, kullanıcı verisi değişikliği ve ekran değişikliği ayrı sorumluluklardır. Bu alıştırmada tümünü bir arada izledik. Migration dosyalarını ve snapshot'ı commit edin; gerçek .db dosyasını commit etmeyin.

Kod kaynağı: [Voku.Web/Data/SiteDbContextFactory.cs:1](#)

SeedData: HTML'den ilk içeriklere

Öğrenme hedefi: İlk açılışta tasarım dosyalarının veritabanı kaydına nasıl dönüştüğünü takip etmek.

1. CreateScope ile DbContext alınır ve MigrateAsync çağrılır. Tablolar hazır olmadan içerik sorgusu yapılmaz.
2. Giriş kontrolü etkinse ve AdminUsers boşsa ilk admin hesabı hazırlanır. Şifre hashlenir. Kontrol kapalıysa bu bölüm atlanır.
3. Pages tablosu tamamen boşsa Content klasöründeki 10 HTML dosyası okunur.
4. Her HTML dosyasının body bölümü alınır. Script etiketleri çıkarılır; görsel yolları /images/... yapılır.
5. .html bağlantıları MVC route'larına çevrilir. Yazı kartları slug adreslerine, alt menü bağlantıları ilgili sayfalara bağlanır.
6. Google Maps bölümü yer tutucuyla değiştirilir. Contact içine gönderimi kapalı form eklenir.
7. Her sayfa ContentPage olarak context'e Add edilir. Blog-post örneği detay olarak işaretlenir.
8. Aynı detay şablonundan üç ilave yazı türetilir. 9 sabit + 4 detay oluşur. SaveChangesAsync ile kayıtlar kalıcı olur.

```
if (!await db.Pages.AnyAsync())
{
    // Dosyaları oku, HTML'i dönüştür, ContentPage nesnelerini ekle.
}
await db.SaveChangesAsync();
```

db.Pages.Local henüz veritabanına kaydedilmemiş, context'in takip ettiği nesneleri de içerir. Seed içindeki detay şablonunun bu koleksiyondan seçilmesi bu yüzden çalışır. Add işlemi ile veritabanına INSERT işleminin aynı anda olmadığını gösteren güzel bir örnektir.

Tekrar başlatmada Pages doluysa içerikler yeniden eklenmez; mevcut düzenlemeler korunur. Ancak bu koşul “eksik her sayfayı tamamla” değildir: tablo kısmen doluysa eksik seed kayıtları da eklenmez. Seed kaynak dosyalarındaki sonraki değişiklikler mevcut satırları güncellemez.

Seed aşamasındaki regex ile script kaldırma işlemi genel bir HTML güvenlik temizleyicisi değildir. Üstelik adminin sonraki HTML kayıtları bu seed dönüşümünden geçmez. HTML düzenleme yetkisinin kimde olduğu ve CSP davranışı ayrı konulardır.

Kod kaynağı: [Voku.Web/Data/SeedData.cs](https://voku.com/Web/Data/SeedData.cs):1

SiteController: ziyaretçiye veri sunmak

Öğrenme hedefi: Aynı view'ın farklı sayfa verileriyle nasıl kullanılabildiğini öğrenmek.

HTTP / route	Action	Render parametreleri
GET /	Home()	slug="home", detail=false
GET /{slug}	Page(string slug)	URL'deki slug, detail=false
GET /blog/{slug}	Detail(string slug)	URL'deki slug, detail=true
/server-error	ServerError()	Problem(...) döndürür; Razor view kullanmaz

```
var page = await db.Pages.AsNoTracking()
    .SingleOrDefaultAsync(p =>
        p.Slug == slug && p.IsDetail == detail && p.Published);

ViewBag.Posts = await db.Pages.AsNoTracking()
    .Where(p => p.IsDetail && p.Published)
    .OrderByDescending(p => p.UpdatedUtc)
    .ToListAsync();
return View("Page", page);
```

Örneğin GET /about isteği Page("about") action'ına ulaşır. Render, Pages tablosundan slug=about, IsDetail=false ve Published=true olan kaydı ister. Sonuç bir ContentPage nesnesidir. Controller bu nesneyi View("Page", page) ile Views/Site/Page.cshtml'e gönderir.

View aramasında controller adı Site olduğundan Page view'ı önce Views/Site altında aranır. Buradaki ikinci page değişkeni dosya adı değil, view'a taşınacak model nesnesidir. Ziyaretçide About.cshtml diye ayrı bir dosya olmaması bu yüzden normaldir.

Kayıt yoksa veya taslaksa Response.StatusCode=404 yapılır; yayınlanmış error sayfası varsa aynı Page view'ında onun içeriği gösterilir. Error içeriği de yoksa NotFound döner. GET /error ise doğrudan mevcut bir sabit sayfayı açtığından normalde 200 döner; bu, bulunamayan bir URL'nin 404 davranışından ayrılır.

ViewBag.Posts tüm yayınlanmış detayları taşır. Mevcut kod bu sorguyu her Render çağrısında yapar; view listeden yalnızca blog sayfasında yararlanır. İleride sorgu yalnızca blog için çalıştırılarak gereksiz veri erişimi azaltılabilir.

Kod kaynağı: [Voku.Web/Controllers/SiteController.cs:1](#)

Page.cshtml: entity'nin ekrana dönüşmesi

Öğrenme hedefi: Model, ViewBag, HTML encoding ve layout seçimini gerçek sayfada görmek.

```
@model ContentPage
@{
    Layout = null;
    var showPosts = Model.Slug == "blog" && !Model.IsDetail;
}
<title>@Model.Title - Voku</title>
<meta name="description" content="@Model.Description">
@Html.Raw(content)
```

Bu örnek kısaltılmıştır. @model ContentPage, bu view'ın beklediği model türünü belirtir. Controller'ın gönderdiği nesne view'da Model adıyla okunur. Title tarayıcı sekmesinde, Description meta etiketinde kullanılır. Görünen h1/h2 başlıkları ise BodyHtml içindeki metinlerdir; Title değişince otomatik değişmez.

Layout=null, kök _ViewStart dosyasının varsayılan _Layout seçimini bu view için iptal eder. Page.cshtml kendi html/head/body yapısını kurar ve Voku CSS/JS dosyalarını yükler. Böylece şablonun gövdesi başka bir MVC layoutunun içine yanlışlıkla iki kez sarılmaz.

Razor normalde @Model.Title gibi metinleri HTML-encode eder. Html.Raw(content) ise gövdedeki HTML'yi markup olarak üretir. Aksi halde ziyaretçi bir başlık yerine ekranda "<h1>..." metnini görürdü. Raw kullanımı HTML içeriğinin güvenilirliğini ayrıca önemli hale getirir; bu bir temizleme fonksiyonu değildir.

```
@foreach (ContentPage post in ViewBag.Posts)
{
    <h3><a href="@post.Url">@post.Title</a></h3>
    <p>@post.Description</p>
}
```

Blog sayfasında BodyHtml içindeki <!-- FOOTER --> işareti bulunarak gövde iki parçaya ayrılır. Dinamik "Tüm yazılar" listesi alt bölümden önce eklenir. İşaret yoksa HTML'nin tamamı ilk parça olur ve liste onun ardından gelir. Bu ayrım yalnızca blog sabit sayfasında yapılır.

Model güçlü türlenmiş tek ContentPage nesnesiyken ViewBag.Posts çalışma zamanında okunan ek listedir. Bunları tek bir SitePageViewModel içinde toplamak gelecekte derleme zamanı kontrolünü güçlendirebilir; mevcut uygulama henüz böyle bir sınıf kullanmaz.

Kod kaynağı: [Voku.Web/Views/Site/Page.cshtml:1](#)

AdminController ve tüm route'lar

Öğrenme hedefi: Hangi action hangi ekranı açıyor, hangi model taşıyor?

Yöntem ve URL	Action → sonuç	Model/veri
GET /admin	Index → Admin/Index.cshtml	List<ContentPage>
GET /admin/pages/{slug}/edit	EditPage → Editor → Admin/Edit.cshtml	Sabit ContentPage
GET /admin/posts/{slug}/edit	EditPost → Editor → Admin/Edit.cshtml	Detay ContentPage
GET /admin/posts/new	New → Admin/Edit.cshtml	Henüz kaydedilmemiş ContentPage
POST /admin/pages/{slug}/edit	SavePage → Save → redirect / aynı view	Formdan ContentPage input
POST /admin/posts/{slug}/edit	SavePost → Save → redirect / aynı view	Formdan ContentPage input
POST /admin/posts/new	Create → Save → redirect / aynı view	Yeni detay formu
GET /admin/login	Login → Login.cshtml veya /admin redirect	LoginModel / giriş kapalıysa view yok
POST /admin/login	Login → doğrulama ve oturum	LoginModel
POST /admin/logout	Logout → cookie temizleme → login redirect	Antiforgery token

Controller'daki `[Route("admin")]` bütün action route'larına ortak önek verir. Action'daki `[HttpGet("posts/new")]` ile birleşince `/admin/posts/new` olur. Editor ve Save private yardımcı metotlardır; URL'den doğrudan çağrılan action değildir.

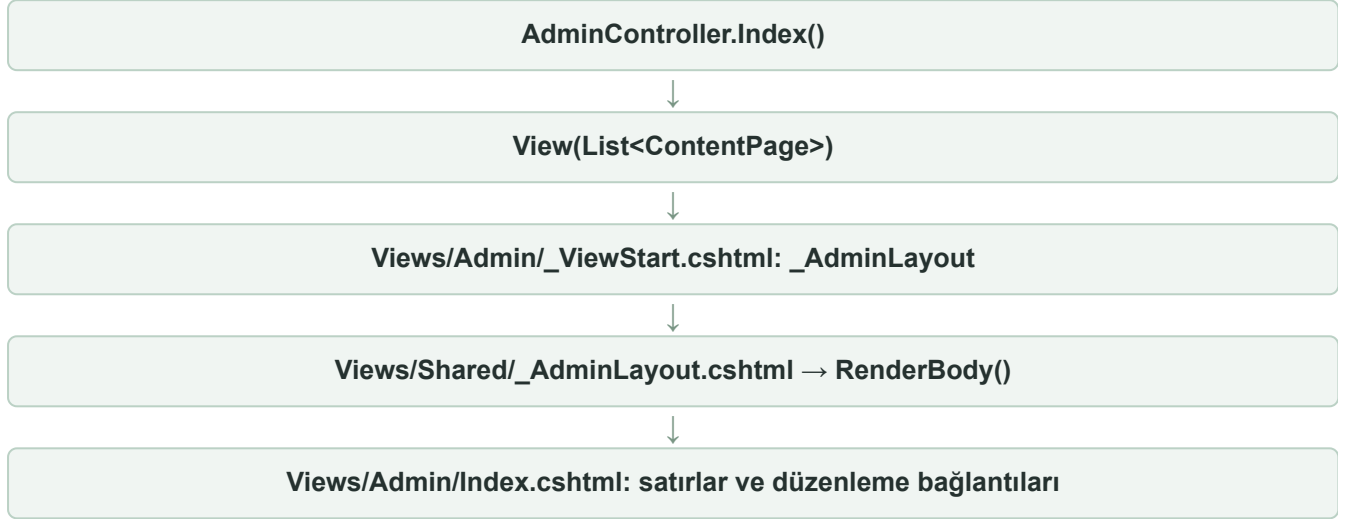
Index tüm sayfaları önce `IsDetail`, sonra Title ile sıralar. View tarafında false ve true grupları ayrı tablolar olarak çizilir. Yayınlanmamış kayıtlar da admin listesinde vardır. Ziyaretçi sorgusundaki Published filtresi admin listesinde kullanılmaz; aksi halde taslakları düzenleyemezdik.

New mevcut detaylardan birinin BodyHtml içeriğini kopyalar. Hiç detay yoksa basit bir main/h1/p gövdesi kullanır. Yeni nesne `IsDetail=true` ve `Published=false` olarak view'a gider. Henüz Add veya SaveChanges yapılmamıştır; formu açıp kapatmak veritabanında yazı oluşturmaz.

Kod kaynağı: [Voku.Web/Controllers/AdminController.cs:1](#)

Admin view'ları ve layout ilişkisi

Öğrenme hedefi: Ortak ekran kabuğu ile sayfaya özgü modelin ilişkisini anlamak.



Kök Views/_ViewStart.cshtml _Layout seçer; Admin klasöründeki daha özel _ViewStart bunu _AdminLayout ile değiştirir. _AdminLayout üst menüyü, siteyi aç bağlantısını, bildirim alanını ve admin.css'i içerir. @RenderBody() tam o isteğin view içeriğinin yerleştiği noktadır.

View	Model	Modelin kullanıldığı yer
Admin/Index	List<ContentPage>	Her satırın Title, Url, Published ve Slug değerleri
Admin/Edit	ContentPage	Title/Slug/Description/Published/Body Html alanları
Admin/Login	LoginModel	Username ve Password inputları
_AdminLayout	Zorunlu özel model yok	User.Identity ve TempData üzerinden ortak ekran durumu

Index view'ı bağlantıyı kayıt türünden üretir: sabit içerikte /admin/pages/about/edit, detayda /admin/posts/ilk-yazim/edit. Böylece aynı ContentPage türü iki farklı edit route'u üzerinden yönetilir. Düzenle action'ı türü URL'den belirler; formun gönderdiği IsDetail değerine dayanmaz.

Views/_ViewImports.cshtml, Voku.Web.Models namespace'ini içeri alır. Bu nedenle view'da uzun namespace yerine ContentPage yazılır. Aynı dosyadaki AddTagHelper kaydı asp-for ve asp-validation-summary özelliklerini etkinleştirir.

TempData["Success"] kaydetme sonrası yönlendirmede mesajı bir sonraki isteğe taşır. Layout bu değeri okuyup "İçerik kaydedildi" bildirimi gösterir. ViewBag aynı istekte view'a veri taşırken TempData yönlendirme sonrasındaki istekte de kullanılabilir. Mevcut kullanıcı kimliği varsa çıkış düğmesi çizilir; anonim kullanımda görünmez.

Kod kaynağı: [Voku.Web/Views/Shared/_AdminLayout.cshtml:1](#)

Kod kaynağı: [Voku.Web/Views/_ViewImports.cshtml:1](#)

Bir admin formunun anatomisi

Öğrenme hedefi: HTML alan adı ile C# özelliği arasındaki bağı görmek.

```
@model ContentPage
<form method="post">
    @Html.AntiForgeryToken()
    <div asp-validation-summary="All"></div>
    <input asp-for="Title" maxlength="160" required>
    <input asp-for="Slug" readonly="@(!Model.IsDetail)" required>
    <textarea asp-for="Description"></textarea>
    <input asp-for="Published">
    <textarea asp-for="BodyHtml" id="BodyHtml"></textarea>
    <button>Değişiklikleri kaydet</button>
</form>
```

Bu, gerçek formun sadeleştirilmiş gösterimidir. Formda action belirtilmediği için POST mevcut URL'ye yapılır. /admin/pages/about/edit ekranı SavePage'e; /admin/posts/new ekranı Create action'ına gönderilir. Aynı Edit view'ı bu sayede üç akışta kullanılabilir.

asp-for="Title" input için name/id ve mevcut değeri üretir. name=Title ile gönderilen değer ContentPage input.Title alanına bağlanır. Label'daki asp-for ilgili inputa bağ kurar. BodyHtml textarea'sı hem gelişmiş HTML editörünün kaynağı hem görsel editörün arka planda güncellediği asıl alandır.

Tarayıcıdan gönderilen alan	Sunucudaki karşılığı
Title=Yeni başlık	input.Title
Slug=ilk-yazim	input.Slug
Description=Kısa açıklama	input.Description
Published=true / false	input.Published
BodyHtml=<main>...	input.BodyHtml
__RequestVerificationToken	MVC antiforgery doğrulaması; entity alanı değildir

Published bool olduğu için Tag Helper checkbox üretir; işaretli olmayan kutuların false olarak bağlanabilmesi için yardımcı hidden değer de üretilebilir. Sunucu tarafında ModelState kontrolü devam eder. Tarayıcıdaki required tek güvence değildir.

Id ve UpdatedUtc düzenlenebilir input olarak sunulmaz. Save gerçek Id'yi bulduğu kayıttan alır, UpdatedUtc'yi sunucuda yeniler. IsDetail route'a göre belirlenir. Bu yaklaşım, istemcinin gönderdiği her özelliği güvenilir kabul edip kaydetmekten kaçınır.

Kod kaynağı: [Voku.Web/Views/Admin/Edit.cshtml:1](#)

Örnek 1: Hakkımızda sayfasını düzenlemek

Öğrenme hedefi: GET → form → POST → EF → redirect → ziyaretçi ekranı zincirini sonuna kadar izlemek.

1. **GET /admin/pages/about/edit:** Routing, AdminController.EditPage("about") action'ını seçer. AdminAccess mevcut ayarda isteğe izin verir.
2. **Editor("about", false):** Pages tablosunda Slug=about ve IsDetail=false kaydı AsNoTracking ile bulunur. Yayın durumu ne olursa olsun admin kayda erişebilir.
3. **View("Edit", page):** ContentPage nesnesi Views/Admin/Edit.cshtml'e taşınır. Inputlar mevcut değerlerle dolar; editor.js BodyHtml'den görsel alanlar üretir.
4. **Kullanıcı değiştirir:** Başlık inputunu veya görsel editörde bir paragrafı değiştirir. Paragraf değişikliği BodyHtml textarea'sına yazılır.
5. **POST aynı URL:** Form verisi ContentPage input nesnesine bağlanır. Antiforgery token ve model doğrulaması denetlenir.
6. **SavePage([FromRoute] slug, input):** URL'deki about mevcut kaydı bulmak için kullanılır. Save sabit route değişimine ve slug çakışmasına bakar.
7. **Alan atamaları:** Title, Slug, Description, BodyHtml ve Published gerçek entity'ye kopyalanır; UpdatedUtc yenilenir. SaveChangesAsync SQLite kaydını günceller.
8. **Redirect:** Başarılı kayıttan sonra /admin/pages/about/edit adresine 302 döner. Tarayıcı yeni GET yapar; layout başarı mesajını gösterir.
9. **GET /about:** SiteController güncel kaydı okur. Published=true ise Page.cshtml yeni HTML'yi ziyaretçiye gösterir.

Bu örnekte iki farklı ContentPage nesnesi düşünün: input, POST verisini taşıyan geçici nesnedir; page, DbContext'in veritabanından yüklediği ve değişiklik takibi yaptığı gerçek kayıttır. Save bunlar arasında açık alan atamaları yapar.

Başarı sonrası yönlendirme, POST yanıtında sayfayı doğrudan çizmek yerine yeni GET başlatır. Böylece kullanıcı başarılı kayıt ekranını yenilediğinde aynı formun yeniden gönderilmesi azaltılır. Hata varsa yönlendirme yapılmaz; aynı input ile Edit view'ı ve hata listesi döner.

Mini deney: Title'ı değiştirin ama BodyHtml'ye dokunmayın. Tarayıcı sekmesi/admin listesi değişirken sayfa içindeki eski başlığın kalabildiğini gözlemleyin. Ardından görsel editörde h1/h2 metnini değiştirin. Böylece metadata ile HTML gövdesinin ayrı saklandığını görürsünüz.

Kod kaynağı: [Voku.Web/Controllers/AdminController.cs:54](#)

Örnek 2: yeni yazı ve slug değişikliği

Öğrenme hedefi: Yeni kayıt oluşturma ile mevcut kaydın adresini değiştirmenin farkını anlamak.

Önce GET /admin/posts/new açılır. New action'ı Published=false olan yeni ContentPage nesnesini Edit view'ına yollar. Title ve Slug girin; örneğin başlık "İlk yazım", slug ilk-yazım. Kopyalanan örnek HTML gövdesini de yazınıza göre düzenleyin.

1. POST /admin/posts/new → Create(input) → Save(null, true, input). oldSlug=null olduğu için yeni entity oluşturulur.
2. Aynı detay slug'ı var mı ve alanlar geçerli mi denetlenir. db.Pages.Add(page), entity'yi eklenecek kayıt olarak işaretler.
3. SaveChangesAsync INSERT yapar. Id veritabanında oluşur. Yeni edit URL'sine yönlendirilir.
4. Taslak bırakıldıysa /blog/ilk-yazım 404 döner. Admin listesinde ise kayıt görünür.
5. Published seçilip kaydedildiğinde ziyaretçi sayfası açılır ve dinamik blog listesine eklenir.

Slug değiştirme: /admin/posts/ilk-yazım/edit formunda Slug alanını ikinci-adres yapın. URL'deki eski slug ile formdaki yeni slug farklı amaç taşır. [FromRoute] özellikle eski adresi okumak için kullanılır.

```
public Task<IActionResult> SavePost(
    [FromRoute] string slug, ContentPage input)
    => Save(slug, true, input);

// slug: "ilk-yazım" - bulunacak kayıt
// input.Slug: "ikinci-adres" - kaydedilecek yeni değer
```

Mevcut kod eski Url'yi alır, slug'ı günceller ve BodyHtml içinde eski URL geçen kayıtları arar. Çift veya tek tırnakla çevrili tam adresler yeni Url ile değiştirilir. Bu genel bir HTML parser değildir; query string, fragment veya farklı yazılmış linkleri kapsayan tam bir bağlantı dönüştürücü de değildir.

Kayıttan sonra edit adresi /admin/posts/ikinci-adres/edit, ziyaretçi adresi /blog/ikinci-adres olur. Eski adrese otomatik 301 yönlendirme yoktur; eski ziyaretçi URL'si 404 verir. Harici sitelerdeki bağlantılar bu yöntemle güncellenemez. Kalıcı yönlendirme geçmişi istenirse ayrıca modellenmelidir.

Kod kaynağı: [Voku.Web/Controllers/AdminController.cs:51](#)

editor.js: görsel alanlar nasıl oluşuyor?

Öğrenme hedefi: Ayrı ayrı görünen metin alanlarının tek BodyHtml değeri olarak nasıl kaydedildiğini öğrenmek.



```

const doc = new DOMParser().parseFromString(source.value, 'text/html');
const sync = () => { source.value = doc.body.innerHTML; };
input.addEventListener('input', () => {
  update(input.value);
  sync();
});
  
```

Sayfa açılınca BodyHtml textarea'sı ve visual-fields alanı bulunur. rebuild(), HTML metnini tarayıcıda ayrı bir document nesnesine çevirir. h1–h6, p, a, span, li ve button elemanlarının yalnızca alt elemanı olmayan ve boş olmayan metinleri için input oluşturulur. İç içe span içeren karmaşık bir paragrafın tek metin kutusuna çevrilmemesinin nedeni yapıyı bozmamaktır.

Bir a etiketi için metin alanına ek olarak href alanı üretilir. Her img için src ve alt alanları üretilir. Alt metni erişilebilir açıklamadır; src dosyanın adresidir. Bu arayüz dosya yüklemeyi; var olan veya erişilebilir bir görsel yolunu yazarsınız.

Üretilen görsel inputlar doğrudan bir EF sütununa ayrı ayrı bağlanmaz. JavaScript'in update callback'i geçici DOM'u değiştirir; sync tüm body'yi yeniden metne çevirip asıl textarea'ya yazar. Sunucu bu tek BodyHtml değerini alır. Dolayısıyla görsel editör için ayrıca JSON API veya ayrı bir kaydetme endpointi yoktur.

Gelişmiş HTML alanı change olayı verdiğinde görsel alanlar yeniden kurulur. Sayfa düzeni, arka plan resimleri veya iç içe HTML gibi görsel editörün kapsamadığı değişiklikler bu alandan yapılır. DOMParser işlemi HTML biçimlendirmesini normalleştirebilir; hedef işlevsel HTML yapısıdır, orijinal boşlukların birebir korunması değil.

Kod kaynağı: [Voku.Web/wwroot/js/editor.js:1](https://voku.web/wwroot/js/editor.js:1)

Ziyaretçi formları ve statik varlıklar

Öğrenme hedefi: Kaydeden admin formu ile bilinçli olarak gönderilmeyen ziyaretçi formunu ayırmak.

```
document.addEventListener('submit', function (event) {
  event.preventDefault();
  event.stopImmediatePropagation();
  // Formun içine bilgilendirme paragrafı eklenir.
}, true);
```

cms.js yalnızca Site/Page.cshtml tarafından yüklenir. submit dinleyicisi capture aşamasında çalışır, normal gönderimi durdurur ve form içinde “Bilgileriniz gönderilmedi ve kaydedilmedi” mesajı gösterir. Bu yüzden e-posta, yorum ya da bülten verisi sunucuda bir tabloya yazılmaz. İletişim için mail servisi ve POST controller action’ı yoktur.

Admin layoutu cms.js yüklemeyiz. Bu nedenle aynı “submit” durdurma kodu admin kaydetme formunu engellemez. Admin/Edit yalnızca editor.js yükler; formun normal POST davranışı devam eder. Kodun hangi sayfada yüklendiği, ne yaptığı kadar önemlidir.

Varlık	İşlev
css/bootstrap/bootstrap.min.css	Voku grid ve temel UI stilleri
css/main.css, style.css, responsive.css	Şablonun genel tasarımı ve ekran uyumu
css/font-awesome.min.css + fonts	İkon fontları
css/admin.css	Admin listesi, form, görsel editör düzeni
css/cms.css	Ziyaretçi form bildirimi ve ufak düzenlemeler
jQuery / own-menu / Bootstrap JS	Şablon menü ve etkileşim altyapısı
Owl Carousel / prettyPhoto	Slider ve görsel açma davranışı
js/cms.js	Eklentileri başlatma, progress genişliği, form durdurma

Orijinal pakette PHP dosyalarının bulunması, uygulamanın PHP kullandığı anlamına gelmez. Sunulan wwwroot’a PHP endpointi eklenmemiştir. Şablonun inline scriptleri seed aşamasında kaldırılmış; kullanılan etkileşimler uygulamanın statik JavaScript dosyalarında başlatılmıştır. Google Maps entegrasyonu da mevcut uygulamada aktif değildir.

JavaScript devre dışıysa form bildirim davranışı çalışmaz; ancak uygulamada e-posta gönderecek bir backend yine yoktur. Bu sürüm “gönderilmedi” geri bildirimi veren bir ön yüz sağlar; iletişim mesajlarını saklayan bir sistem değildir.

Kod kaynağı: Voku.Web/wwwroot/js/cms.js:1

Giriş kontrolü ve mevcut güvenlik davranışı

Öğrenme hedefi: Şimdilik kapalı giriş kontrolünün kodda nasıl korunup ayarlanabildiğini görmek.

```
options.AddPolicy("AdminAccess", policy =>
    policy.RequireAssertion(context =>
        !builder.Configuration.GetValue("Admin:RequireAuthentication", true)
        || context.User.Identity?.IsAuthenticated == true));
```

Politikanın mantığı: giriş kontrolü kapalıysa izin ver; açıksa kimliği doğrulanmış kullanıcı iste. AdminController üzerindeki Authorize bu politikayı seçer. Şimdilik appsettings.json false verdiği için /admin ve düzenleme işlemleri anonim erişime açıktır. Bu durum yalnızca login ekranını gizlemek değildir; yetki politikasının izin vermesidir.

1. Kontrol açıkken GET /admin/login bir LoginModel ile Login.cshtml açar. POST Username ve Password değerlerini modele bağlar.
2. AdminUsers içinde kullanıcı adı aranır. PasswordHasher saklanan hash ile girilen şifreyi doğrular.
3. Başarılı girişte kullanıcı adı claim'i olan ClaimsPrincipal kurulur ve SignInAsync oturum cookie'si oluşturur.
4. Sonraki isteklerde UseAuthentication cookie'den kimliği oluşturur; AdminAccess kontrolü geçerli.
5. Logout POST isteği SignOutAsync ile cookie oturumunu kapatır. Giriş kapalı modda login action'ı doğrudan /admin'e yönlendirir.

Cookie adı Voku.Admin, HttpOnly ayarı true ve tanımlı bilet süresi dört saattir. Login POST için IP adresi temelinde dakikada 10 istek sınırı vardır; sınır aşırsa 429 döner. Bu, ayrı bir hesap bazlı kilitleme sistemi değildir. Rol tabanlı çok kullanıcıli yetki modeli de bulunmaz.

Antiforgery/CSRF kontrolü, giriş kapalı olsa bile yazma isteklerinde sürer. Token, formun beklenen istemci akışından geldiğini denetlemeye yardımcı olur; kullanıcı yetkisinin yerine geçmez. Anonim kullanıcı geçerli formu açıp token alabiliyorsa mevcut ayarda düzenleme yapabilir.

CSP inline scriptleri engeller ve kaynakları sınırlar; Html.Raw içeriğini temizleyen bir sanitizier değildir. HTML editörünün güvenilir yöneticiler için tasarlandığını unutmamak gerekir. Yayın ortamında kimlerin düzenleme yapabileceği bu ayara bağlıdır.

Kod kaynağı: [Voku.Web/Program.cs:15](#)

Kod kaynağı: [Voku.Web/Controllers/AdminController.cs:19](#)

Testler: davranışı nasıl doğruluyoruz?

Öğrenme hedefi: Kodun derlenmesi ile gerçek akışların çalışmasının farkını anlamak.

dotnet build C# ve Razor derleme hatalarını yakalar. Bir route yanlış kayıt buluyor veya kaydetme sonrası veri kayboluyorsa derleme yine başarılı olabilir. tests/smoke.py bu nedenle uygulamayı gerçek HTTP istekleriyle sınar.

1. Python standart kütüphanesiyle kullanılabilir yerel port seçilir. Geçici klasörde test.db yolu hazırlanır.
2. Rastgele test şifresi ve ConnectionStrings__Default, uygulama alt sürecine ortam değişkeni olarak verilir. Gerçek App_Data/voku.db kullanılmaz.
3. Derlenmiş DLL dotnet ile başlatılır. Site açılınca sayfalar, CSS ve bir görsel kontrol edilir.
4. CookieJar oturum/antiforgery cookie'lerini saklar. Login formundan token okunup POST edilir. Yönlendirmeler otomatik izlenmeyerek 302 doğrudan test edilir.
5. İçerik değişikliği, taslak/yayın, slug çıkışması, geçersiz slug, slug değişince link düzeltme ve CSRF doğrulanır.
6. Sunucu yeniden başlatılır; değişen içerik hâlâ mevcut mu kontrol edilir.
7. İkinci, boş veritabanında giriş kapatılıp şifresiz startup, /admin erişimi ve anonim kayıt sınanır.

Kontrol	Beklenen sonuç
Oturumsuz /admin, giriş açık	302 login yönlendirmesi
Token olmadan yazma	400
Taslak veya bilinmeyen ziyaretçi sayısı	404
Başarılı admin kayıt	302 ve ardından değişen içerik
Çakışan/geçersiz slug	200 ile hata içeren edit formu; yeni kayıt yok
POST /contact	405; uygun POST action'ı yok
Giriş kapalı /admin	200; cookie oturumu gerekmiyor

Test 13 başlangıç sayfasına bir detay eklediği için ilk test veritabanında kayıt sayısı 14 bekler. Mevcut smoke testi JavaScript çalıştırmaz; görsel editörün input davranışını veya responsive görünümü bu test tek başına kanıtlamaz. Bunlar için tarayıcı testi gerekir. Önceki geliştirmede tarayıcı kontrolleri de yapılmıştır; repodaki CI tanımı HTTP smoke testini çalıştırır.

Kod kaynağı: [tests/smoke.py:1](#)

GitHub, CI ve yayınlama

Öğrenme hedefi: Kaynak kodun saklanması, test edilmesi ve yayınlanmasını ayırmak.

```
dotnet restore Voku.slnx
dotnet build Voku.slnx --no-restore
python3 tests/smoke.py

# Yayın çıktısı oluşturur; sunucuya yükleme yapmaz.
dotnet publish Voku.Web -c Release -o ./publish
```

GitHub Actions, main dalına push ve pull request olaylarında çalışacak şekilde tanımlanmıştır. Ubuntu ortamında repository alınır, .NET 10 ve Python kurulur; restore, build, smoke.py sırasıyla çalıştırılır. Bu dosyanın varlığı son workflow koşusunun başarılı olduğunu tek başına kanıtlamaz; koşu sonucu GitHub Actions ekranından görülür.

Commit türü	Anlamı	Bu projeden kullanım
chore	Altyapı / yardımcı işler	.NET MVC başlangıcı ve paketler
feat	Yeni davranış	Görsel editör, ayarlanabilir admin erişimi
fix	Mevcut davranış düzeltmesi	Kartları doğru slug sayfalarına bağlama
test	Doğrulama kodu	İzole entegrasyon testleri
docs	Açıklama ve rehber	README ve bu eğitim belgesi

Git commit yerel sürüm kayıdır; git push bunu origin uzak deposuna taşır. Çalışan SQLite dosyası, bin ve obj çıktıları .gitignore ile hariç tutulur. Bu nedenle başka makineye clone yapıldığında mevcut yerel içerik veriniz otomatik gelmez; ilk açılışta seed içerikleri oluşur. Gerçek içerikleri taşımak için veritabanı ayrıca yedeklenip aktarılır.

Publish çıktısı uygulamanın çalışması için dosyaları hazırlar. Veritabanı için kalıcı/yazılabilir konum, bağlantı ayarı, HTTPS ve ortam değişkenleri yayın hedefinde ayrıca düzenlenir. Mevcut başlangıç migrationı tek örnekli kullanım için basittir; çok örnekli dağıtımda migration uygulamasını kontrollü bir dağıtım adımına ayırmak değerlendirilebilir.

SQLite WAL modunda çalışabilir. Açık veritabanını yedeklerken yalnızca .db dosyasını rastgele kopyalamak yerine SQLite yedekleme yöntemi kullanmak veya uygulamayı düzgün durdurup tutarlı yedek almak gerekir. Veritabanını Git'e eklemek yedekleme stratejisi değildir.

Kod kaynağı: [.github/workflows/ci.yml:1](#)

Kod kaynağı: [.gitignore:1](#)

Hata ayıklama ve sık karşılaşılan durumlar

Öğrenme hedefi: Belirtiyi ilgili katmana bağlayarak sorunu daraltmak.

Belirti	Olası neden / izlenecek yer
Voku.Web yolu bulunamıyor	Terminal konumu yanlış. Mutlak csproj yolu kullanın.
İlk çalıştırma şifresi hatası	Giriş etkin, AdminUsers boş. Ayarı veya ilk şifre değişkenini kontrol edin.
Admin doğrudan açılmıyor	Ortam değişkeni JSON false değerini override ediyor olabilir; çalışan süreci yeniden başlatın.
Yeni içerik ziyaretçide 404	Published, Slug ve IsDetail üçlüsünü kontrol edin.
Title değişti ama sayfa başlığı değişmedi	Görünür başlık BodyHtml içindedir. Title metadata/listede kullanılır.
Content/about.html değişikliği görünmüyor	Seed yalnızca tablo boşken çalışır. Mevcut içerik admin üzerinden güncellenir.
Admin POST 400	Antiforgery token/cookie eksik olabilir. Formu yeniden GET ile açın.
Kaydetme 200 ama değişiklik yok	Edit view hata ile dönmüş olabilir; validation summary'ye bakın.
Slug değişince eski link 404	Eski adrese redirect geçmişi tutulmuyor. Yeni adresi kullanın.
Görsel görünmüyor	/images/... yolu ve wwwroot dosyası eşleşiyor mu? Tarayıcı Network sekmesinden isteği inceleyin.
Model alanı veritabanında yok	Migration üretildi mi, build edildi mi ve doğru veritabanına uygulandı mı?
Port kullanımda	Başka uygulama süreci aynı portu dinliyor olabilir. Eski süreci durdurun veya profil portunu değiştirin.

İstek izleme alışkanlığı: Tarayıcı geliştirici araçlarında Network'ten URL, HTTP yöntemi, durum kodu ve form payload'unu görün. Sonra bu URL'ye karşılık gelen controller action'ında breakpoint koyun. input ve veritabanından gelen page nesnesini ayrı inceleyin. Hatanın view, binding, doğrulama veya kayıt adımlarından hangisinde olduğunu böyle daraltabilirsiniz.

EF günlüklerinde SELECT görmek okuma, UPDATE görmek değişiklik, INSERT görmek yeni kayıt belirtisidir. Her görünen SQL satırı hata değildir. Asıl hata için exception türü ve stack trace'teki proje dosyası/satırı incelenir. Şifre/hash gibi verileri teşhis amacıyla loglamayın.

Kod kaynağı: [Voku.Web/Controllers/AdminController.cs:62](#)

Uygulamalı çalışma planı ve cevaplar

Öğrenme hedefi: Dosya isimlerini ezberlemek yerine veri akışını açıklayabilmek.

Alıştırma	Beklenen gözlem / cevap
/about açılınca hangi view?	SiteController.Page → Render → Views/Site/Page.cshtml; model ContentPage.
/admin listesinde hangi model?	AdminController.Index → List<ContentPage> → Admin/Index.cshtml.
Yayın işaretini kaldırın	Admin kaydı korur ve gösterir; ziyaretçi sorgusu Published şartıyla kaydı dışlar.
Aynı slug ile iki detay oluşturun	Controller hatası ve veritabanı unique indeksi aynı türde çakışmayı engeller.
Textarea BodyHtml ile bir metni değiştirin	Kayıtta HTML string'i güncellenir; Html.Raw ile ziyaretçi ekranına çıkar.
Yeni görsel src girin	Ayrı görsel tablosu oluşmaz; BodyHtml içindeki img src değişir.
Program.cs'de UseStaticFiles olmasa?	CSS/JS/görsel isteklerinin mevcut statik sunum yolu çalışmaz.
SaveChangesAsync kaldırılırsa?	Nesnelerin bellekte değişmesi veritabanına yazılmış oldukları anlamına gelmez.
Yeni özellik sadece modelde eklenirse?	Mevcut veritabanında sütun garanti edilmez; migration gerekir.
Admin şifresini ortam değişkeniyle değiştirin	Mevcut hesap varsa seed parolayı yenilemez; ilk kurulum ayarı reset değildir.

Geliştirme önerileri — mevcut özellikler değildir: Ayrı view/input modelleri; HTML için açık güven modeli ve sanitization; güncelleme geçmişi; slug redirect tablosu; görsel yükleme; yazar/kategori ilişkileri; pagination; çakışan editleri algılayan concurrency kontrolü. Her ekleme yeni tablo gerektirmez, ancak her biri veri yolu ve test açısından tasarlanmalıdır.

Mevcut sürümde silme endpointi, gerçek iletişim mesajı kaydı, SMTP gönderimi, tam WYSIWYG editör, içerik versiyonlama ve geri alma geçmişi yoktur. New action'ı mevcut bir detayın gövdesini kopyalar; sabit ve değişmez bir şablon kütüphanesi üzerinden çalışmaz. Blog gövdesindeki örnek kartlar ile dinamik yazı listesi de ayrı içerik kaynaklarıdır.

Kendinizi sınavın: “Bu input hangi özelliğe bağlanır, bu özellik hangi entity'ye kopyalanır, hangi tabloya yazılır ve hangi view'da yeniden görünür?” sorusunu bir alan için baştan sona cevaplayabiliyorsanız MVC veri akışını anlamışsınızdır.

Terimler ve hızlı dosya rehberi

Öğrenme hedefi: Koda dönerken kavramları ve ilgili dosyaları hızla bulmak.

Terim	Bu projedeki anlamı
Entity	Veritabanına eşlenen nesne: ContentPage, AdminUser.
View model / form model	Ekrana/form girişine uygun veri nesnesi; LoginModel örneği.
Action	HTTP endpointinin yürüttüğü controller metodu.
Route	HTTP yöntemi ve URL'nin action ile eşleşme kuralı.
Model binding	Form/route verilerinin C# parametre ve nesnelere aktarılması.
ModelState	Binding ve doğrulama hataları ile gönderilen değerlerin durumu.
DbContext	EF sorgu, eşleme ve değişiklik takibi birimi.
Migration	Veritabanı şema değişikliğinin sürümlenmiş C# tarifi.
Seed	Boş veritabanına ilk örnek kayıtların eklenmesi.
Layout	View içeriğinin yerleştiği ortak ekran kabuğu.
Tag Helper	asp-for gibi özelliklerden sunucuda HTML üretme mekanizması.
Slug	URL'de kullanılan okunabilir içerik tanımlayıcısı.
Middleware	İstek/yanıt işleme hattındaki bileşen.
DI	Gerekli servislerin framework tarafından nesneye verilmesi.
Tracking	EF'nin yüklenen entity değişikliklerini izlemesi.
CSRF token	Yazma isteklerinde form/cookie doğrulamasının parçası.

Okuma sırası önerisi: **Program.cs** → **ContentPage.cs** → **SiteDbContext.cs** → **SeedData.cs** → **SiteController.cs** → **Site/Page.cshtml** → **AdminController.cs** → **Admin/Edit.cshtml** → **editor.js** → **smoke.py**. Bu sıra başlangıçtan veriye, veriden ekrana, ekrandan tekrar kayda ilerler.

Bu belgedeki “Kod kaynağı” bağlantıları anlatılan sürümün GitHub dosyalarına gider. Sonradan kod değişirse güncel davranışı karşılaştırmak için bağlantıdaki commit ile kendi çalışma dalınızı kontrol edin. Yerel mutlak yollar size ait mevcut çalışma klasörüne göre verilmiştir.

Kitabın ana akışı: İstek → route → controller → EF sorgusu → entity → Razor view → HTML formu → model binding → doğrulama → SaveChangesAsync → yeni istek. Voku projesindeki ekranların büyük bölümü bu döngünün farklı örnekleridir.